

Lógica de Predicados

Lógica Computacional · 3º Semestre | Engenharia de Software

Prof. Me. Keny Lucas da Silva Goes



CAMPUS XXIII
PARAUAPEBAS

Ponte com a Aula 2

Lógica Proposicional (Aula 2)

A sentença inteira era representada por uma única letra.

P = "João é aluno"

A estrutura interna da sentença era ignorada.

Lógica de Predicados (Aula 3)

Agora analisamos **o que está dentro** da sentença: quem é o sujeito e qual propriedade lhe é atribuída.

Aluno(João)

Predicado: **Aluno** · Indivíduo: **João**

Limitação da Lógica Proposicional

Na lógica proposicional, cada sentença é tratada como um átomo opaco — não conseguimos ver o que há dentro dela.

Representação proposicional

P = "Maria é estudante"

Q = "João é estudante"

P e Q parecem completamente diferentes, mas **escondem a mesma propriedade**: "ser estudante".

Problema

Não conseguimos expressar que Maria e João **compartilham uma propriedade comum**, nem raciocinar sobre "todos os estudantes" ou "algum estudante".

O que muda com a Lógica de Predicados

A lógica de predicados permite representar **objetos**, **propriedades** e **relações** de forma explícita e estruturada.

Propriedade unária

Estudante(**Maria**)

Estudante(**João**)

Relação binária

MaiorQue(**10**, **5**)

Conecta dois elementos entre si

Vantagem

Podemos raciocinar sobre **classes inteiras** de objetos com quantificadores

O que é um Predicado?

Um **predicado** é uma expressão que representa uma **propriedade** ou **relação**. Ele se torna verdadeiro ou falso quando aplicado a indivíduos concretos do domínio.

Aluno(x)

"x é aluno"

Par(x)

"x é par"

Acessa(x, sistema)

"x acessa o sistema"

Predicado como Propriedade

Predicados com **uma variável** expressam propriedades de um único indivíduo. O **x** representa o indivíduo que está sendo analisado.



Aluno(x)

"x é aluno"



Ativo(x)

"x está ativo"



Par(x)

"x é par"



Aprovado(x)

"x foi aprovado"

Predicado como Relação

Predicados com **duas ou mais variáveis** expressam relações entre elementos. São fundamentais para modelar permissões, comparações e vínculos em sistemas.

MaiorQue(x, y)

"x é maior que y" — relação de ordem entre dois valores

Matriculado(x, disciplina)

"x está matriculado em disciplina" — relação aluno ↔ curso

Permite(usuario, recurso)

"usuario tem permissão para acessar recurso" — relação de controle de acesso

Variáveis na Lógica de Predicados

Variáveis representam elementos não especificados do domínio. Funcionam como **espaços reservados** que podem assumir qualquer valor do conjunto considerado.

Símbolos comuns: **x, y, z, w**

$\text{Aluno}(x) \rightarrow \text{"x é aluno"}$

x pode ser qualquer pessoa do domínio

Genérica

Representa qualquer elemento – seu valor é indefinido até ser quantificado ou instanciado

Flexível

Permite formular regras gerais: "Para todo x, se x é usuário, então x tem senha"

Essencial

Variáveis são o que tornam os predicados reutilizáveis e expressivos

Constantes na Lógica de Predicados

Constantes representam elementos **específicos e fixos** do domínio — um indivíduo concreto, não genérico.

Exemplos de constantes: **joao, maria, logicaComputacional**

Aluno(**maria**) → "Maria é aluna" — pode ser avaliado como V ou F

Aluno(maria)

Predicado aplicado à constante **maria**

Matriculado(joao, logicaComputacional)

Relação entre dois indivíduos específicos

Variáveis x Constantes

Elemento	Representa	Exemplo
Variável	Elemento genérico, indefinido	x, y, z
Constante	Elemento específico, fixo	maria, joao
Predicado com variável	Sentença aberta (sem valor lógico)	Aluno(x)
Predicado com constante	Sentença fechada (tem valor lógico)	Aluno(maria)

Sentença Aberta

Uma **sentença aberta** contém pelo menos uma **variável livre** — uma variável que não está ligada a nenhum quantificador.

Por isso, ela **não pode ser avaliada** como verdadeira ou falsa sem que saibamos quem é **x**.

Aluno(x)

Quem é x? Enquanto não soubermos, a sentença permanece aberta.

Se x = "Maria"

Aluno(Maria) → pode ser
Verdadeiro

Se x = "Carro"

Aluno(Carro) →
provavelmente **Falso**

Sentença Fechada

Uma **sentença fechada** pode ser avaliada como verdadeira ou falsa — ou porque usa constantes, ou porque as variáveis estão **quantificadas**.

Com constante

Aluno(maria)

"Maria é aluna" $\rightarrow$ V ou F

Com quantificador universal

$\forall x$ Aluno(x)

"Todo x é aluno" $\rightarrow$ V ou F

Com quantificador existencial

$\exists x$ Aluno(x)

"Existe x que é aluno" $\rightarrow$ V ou F

Domínio de Interpretação

O **domínio de interpretação** é o conjunto de elementos sobre os quais uma sentença será analisada. Ele define o **universo de discurso** da fórmula.

Domínio = {estudantes da turma}

Aluno(x) $\rightarrow$ "x é aluno desta turma"

O domínio restringe

Apenas elementos do domínio podem ser substituídos nas variáveis

O domínio define sentido

Sem saber o domínio, não sabemos sobre quem estamos falando

O domínio altera o valor

A mesma sentença pode ser V em um domínio e F em outro

A Importância do Domínio

O **valor lógico** de uma proposição quantificada depende diretamente do domínio escolhido. Veja o mesmo enunciado em dois contextos:

$\forall x$ Estudante(x)

Domínio: alunos matriculados na turma

→ Provavelmente **Verdadeiro**, pois todos na turma são estudantes

$\forall x$ Estudante(x)

Domínio: habitantes da cidade

→ Provavelmente **Falso**, pois nem todos os cidadãos são estudantes

Domínio em Sistemas Computacionais

Em sistemas reais, o domínio costuma ser um **conjunto de registros** — usuários, produtos, transações. Predicados funcionam como **filtros e regras de validação**.

Domínio: Usuários cadastrados no sistema



Ativo(x)

x é um usuário ativo



Autenticado(x)

x realizou login com sucesso



Administrador(x)

x possui privilégios de administrador

Quantificadores

Os quantificadores transformam **sentenças abertas** em **proposições quantificadas** — com valor lógico definido sobre um domínio.

$\forall$ — Universal

"Para todo x..."

A propriedade vale para **todos** os elementos do domínio



$\exists$ — Existencial

"Existe pelo menos um x tal que..."

A propriedade vale para **ao menos um** elemento

Quantificador Universal — $\forall$

O símbolo $\forall$ (lê-se "para todo") afirma que uma propriedade vale para **cada elemento** do domínio sem exceção.

$\forall x P(x)$

"Para todo x , $P(x)$ é verdadeiro"

Basta **um único elemento** que não satisfaça $P(x)$ para tornar a proposição **falsa**.

Exemplo

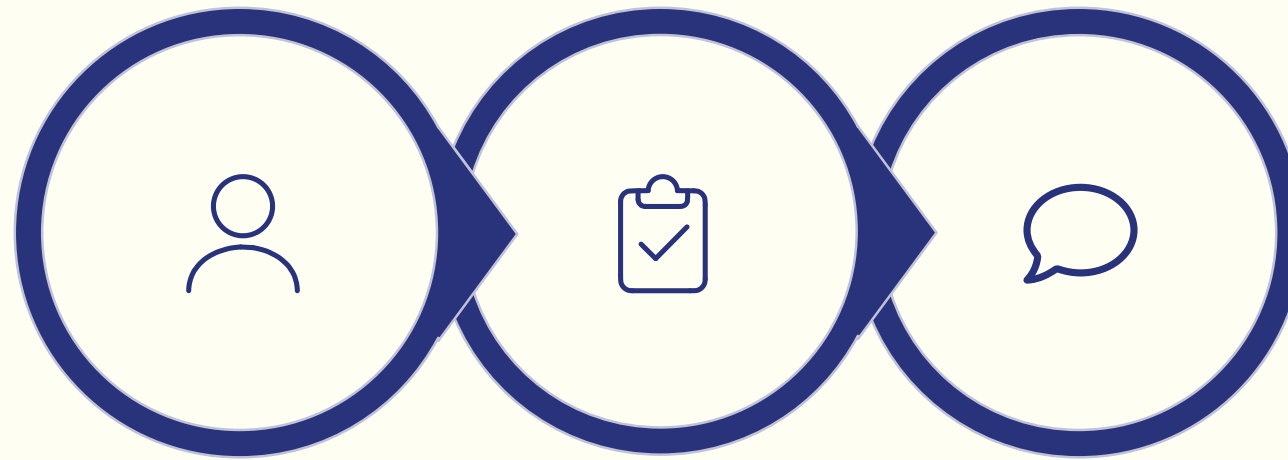
$\forall x \text{ Aluno}(x)$

"Todo elemento do domínio é aluno"

Leitura em português

"Para todo x , x é aluno"

Exemplo: Quantificador Universal



Domínio

Predicado

Proposição

Leitura: "Todos os alunos da turma estão matriculados em Lógica Computacional." — Esta proposição é verdadeira somente se **nenhum** aluno da turma deixar de estar matriculado.

Quando o Universal é Falso?

Uma proposição universal é **falsificada por um único contraexemplo**.

Proposição

$\forall x \text{ Ativo}(x)$

"Todos os usuários estão ativos"

Contraexemplo

Existe usuário **inativo** no sistema $\rightarrow$
proposição **Falsa**

Implicação

Para provar que $\forall x P(x)$ é **falsa**, basta encontrar **um x** tal que $\neg P(x)$ seja verdadeiro

Quantificador Existencial — $\exists$

O símbolo $\exists$ (lê-se "existe") afirma que **ao menos um elemento** do domínio satisfaz a propriedade.

$\exists x P(x)$

"Existe pelo menos um x tal que $P(x)$ é verdadeiro"

Basta **um único elemento** que satisfaça $P(x)$ para tornar a proposição **verdadeira**.

Exemplo

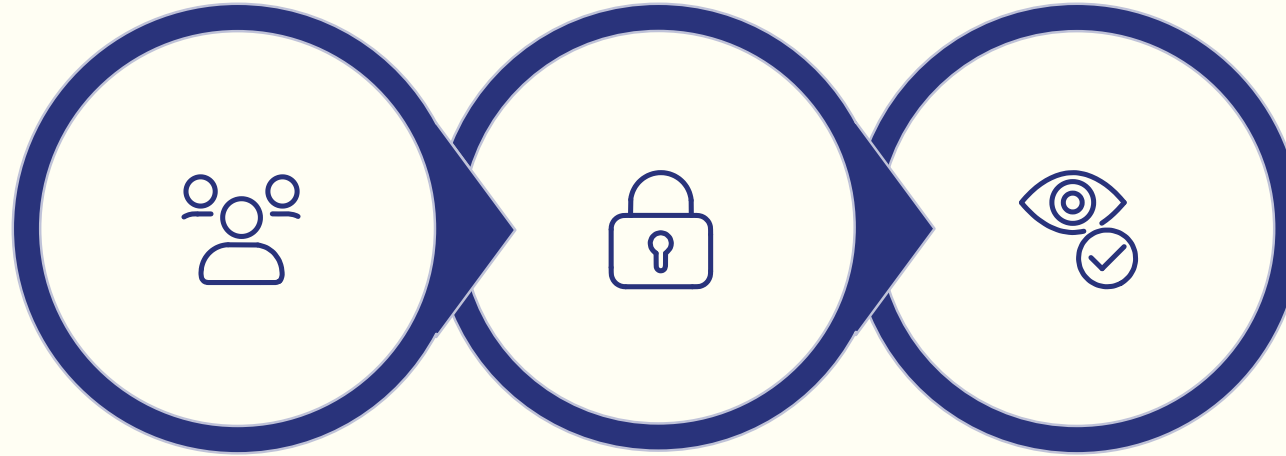
$\exists x \text{ Administrador}(x)$

"Existe pelo menos um administrador"

Leitura em português

"Para algum x , x é administrador"

Exemplo: Quantificador Existencial



Domínio

Predicado

Proposição

Leitura: "Existe pelo menos um usuário bloqueado no sistema." — Basta encontrar **um único usuário** com status bloqueado para que esta proposição seja verdadeira.

Quando o Existencial é Falso?

Uma proposição existencial é **falsa quando nenhum elemento** do domínio satisfaz o predicado.

Proposição

$\exists x \text{ Bloqueado}(x)$

"Existe usuário bloqueado"

Condição de falsidade

Todos os usuários estão liberados $\rightarrow$
proposição **Falsa**

Implicação

Para provar que $\exists x P(x)$ é **falsa**, é preciso verificar **todos os elementos** e confirmar que nenhum satisfaz $P(x)$

Universal x Existencial

Símbolo	Leitura	Verdadeiro quando	Falso quando
$\forall x P(x)$	"Para todo x, P(x)"	Todos satisfazem P	Pelo menos um não satisfaz
$\exists x P(x)$	"Existe x tal que P(x)"	Pelo menos um satisfaz P	Nenhum satisfaz P

 $\forall$ exige **todos** · $\exists$ exige **ao menos um**

Traduzindo Sentenças para Lógica de Predicados

Sentença com "todo"

"Todo aluno está matriculado."

$\forall x \text{ Aluno}(x) \rightarrow \text{Matriculado}(x)$

O quantificador universal combina com **implicação ($\rightarrow$)**: "se x é aluno, então x está matriculado".

Sentença com "existe"

"Existe um aluno aprovado."

$\exists x \text{ Aluno}(x) \wedge \text{Aprovado}(x)$

O quantificador existencial combina com **conjunção ($\wedge$)**: "existe x que é aluno e está aprovado".

Cuidado com a Tradução

A escolha entre $\forall$ e $\exists$ muda completamente o sentido da frase – e o conectivo que os acompanha também importa.

$\forall x \text{ Aluno}(x) \rightarrow \text{Aprovado}(x)$

"Todo aluno é aprovado"

Afirmação sobre **todos** os alunos sem exceção

$\exists x \text{ Aluno}(x) \wedge \text{Aprovado}(x)$

"Existe aluno aprovado"

Afirmação sobre **pelo menos um** aluno específico



Nunca use $\exists x P(x) \rightarrow Q(x)$ – isso raramente captura o sentido desejado. Com $\exists$, prefira $\wedge$.

Quantificadores e Conectivos Lógicos

Na prática, quantificadores quase sempre aparecem combinados com **conectivos lógicos** para expressar condições mais ricas.

$\forall$ com implicação

$\forall x \text{ Usuario}(x) \rightarrow \text{Ativo}(x)$

"Todo usuário está ativo"

$\exists$ com conjunção

$\exists x \text{ Usuario}(x) \wedge \text{Bloqueado}(x)$

"Existe usuário bloqueado"

$\forall$ com negação

$\forall x \text{ Usuario}(x) \rightarrow \neg \text{Bloqueado}(x)$

"Nenhum usuário está bloqueado"

Negação de Proposições Quantificadas

Negar uma proposição quantificada não é simplesmente colocar um $\neg$ na frente — o tipo de quantificador também muda.

Negar "todos"

Gera "existe pelo menos um que **não**"

$$\neg \forall \rightarrow \exists \neg$$

Negar "existe"

Gera "todos **não**" (nenhum)

$$\neg \exists \rightarrow \forall \neg$$

Regra: Negação do Universal

Negar uma proposição universal transforma o quantificador $\forall$ em $\exists$ e **nega o predicado**.

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

Proposição original

"Todos os usuários estão ativos."

$$\forall x \text{Ativo}(x)$$

Negação

"Existe pelo menos um usuário que **não** está ativo."

$$\exists x \neg \text{Ativo}(x)$$

Regra: Negação do Existencial

Negar uma proposição existencial transforma o quantificador $\exists$ em $\forall$ e **nega o predicado**.

$$\neg \exists x P(x) \equiv \forall x \neg P(x)$$

Proposição original

"Existe usuário bloqueado."

$\exists x \text{Bloqueado}(x)$

Negação

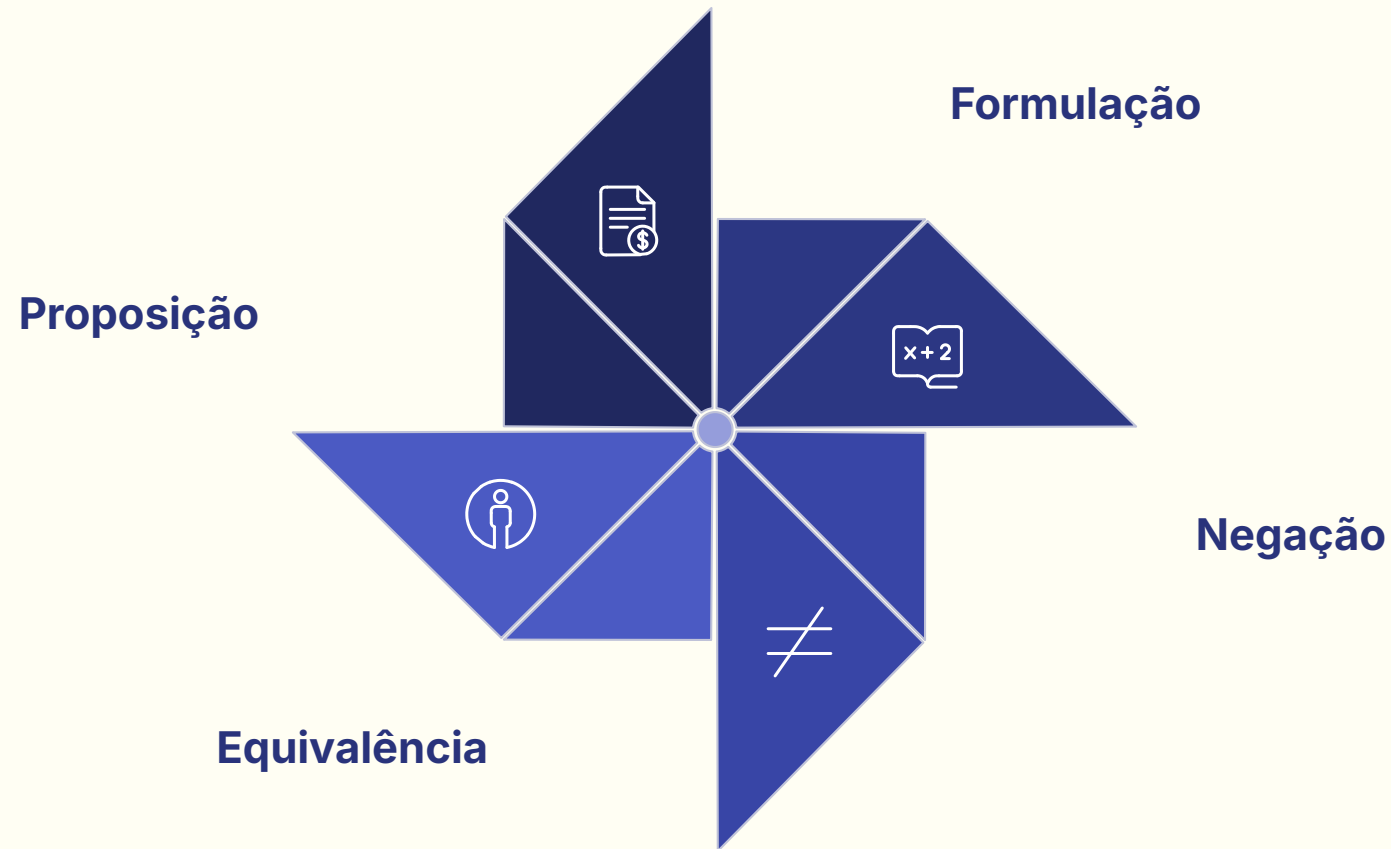
"Nenhum usuário está bloqueado" / "Todos os usuários **não** estão bloqueados."

$\forall x \neg \text{Bloqueado}(x)$

Exemplo Comparativo de Negação

Etapa	Forma lógica	Leitura em português
Proposição original	$\forall x \text{ Entregou}(x)$	"Todos os alunos entregaram a atividade"
Negação aplicada	$\neg \forall x \text{ Entregou}(x)$	"Não é verdade que todos entregaram"
Forma equivalente	$\exists x \neg \text{Entregou}(x)$	"Existe pelo menos um aluno que não entregou"

Negação em Sistemas Computacionais



Aplicação prática: Em sistemas de segurança, basta **um usuário não autenticado** para que a regra de acesso universal falhe e um alerta seja disparado. A negação formaliza exatamente essa lógica de detecção de exceção.